Activity 1

Activity Title: Design a Restaurant Menu Board
Activity Type: Online

Description:
A one-page restaurant menu board that displays food categories and meal cards in a clean grid layout. Students will create sections such as Burgers, Pizza, Drinks, and Desserts, then use CSS Grid to organize the menu items with rows, columns, gaps, and named grid areas.

Objective:
By the end of this activity, students will be creating a restaurant menu webpage using CSS Grid. The page will use `display: grid`, `grid-template-columns`, `grid-template-rows`, `gap`, `grid-template-areas`, and `grid-column` / `grid-row` for positioning. Students will also use Grid Inspector to check the layout visually and understand why Grid is better than Flexbox for this type of structured two-dimensional layout.

Steps:

1. **Set up the page:**
   Make a new HTML file and link an external CSS file. Add a clear page title such as "Fresh Bite Menu".
2. **Add the menu header:**
   Create a header with the restaurant name, a short slogan, and a small opening-hours line.
3. **Create the main menu layout:**
   Add a main container that will hold the menu board sections.
4. **Add menu areas:**
   Create sections for `special`, `burgers`, `pizza`, `drinks`, and `desserts`.
5. **Add food cards:**
   Inside each section, add meal cards with a food name, short description, price, and optional image with meaningful alt text.
6. **Turn the menu board into a Grid:**
   Use `display: grid`, define columns and rows, and add `gap` for clean spacing between sections.
7. **Use grid-template-areas:**
   Create a named layout map so the special offer appears large, while other categories sit around it.

8. **Assign each section to its area:**
   Use matching `grid-area` names for every menu section and check that the names match exactly.
9. **Position one section manually:**

   Use grid-column or grid-row to make the special offer section span more space in the main menu grid.

10. **Use Flexbox inside cards only:**
    Use Flexbox inside the food cards to align the price or button neatly at the bottom.
11. **Inspect the layout:**
    Open DevTools in Chrome or Firefox, enable Grid Inspector, and check the grid lines, gaps, rows, columns, and named areas.
12. **Test and polish:**
    Make sure spacing is consistent, menu text is readable, images fit inside cards, and the layout clearly shows that Grid controls the main structure.

Duration: 35 minutes

Activity 2

Activity Title: Create a Smart Event Dashboard
Activity Type: Online

Description:
A one-page event dashboard that displays upcoming workshops, reminders, and featured sessions in a card-based grid. Some cards will be large and some will be normal size, allowing students to explore auto-placement, `grid-auto-flow`, implicit rows, and the difference between Grid and Flexbox.

Objective:
By the end of this activity, students will be creating an event dashboard using CSS Grid auto-placement. Students will use `repeat()`, `fr`, `minmax()`, `gap`, `grid-auto-rows`, and `grid-auto-flow`. They will also use Grid Inspector to understand how the browser places items automatically and decide when Grid is more suitable than Flexbox for two-dimensional layouts.

Steps:

1. **Set up the files:**
   Create an HTML file and an external CSS file. Add a page title such as "My Event Dashboard".
2. **Add the page header:**
   Create a header with a title and a short intro, for example: "Track your workshops, tasks, and reminders in one place."
3. **Create the dashboard container:**
   Add a main dashboard section that will contain all event cards.
4. **Build event cards:**
   Add at least eight cards. Each card should include a category, title, short description, date or time, and a small action link like "View details".
5. **Make different card types:**
   Give some cards special classes like `featured`, `wide`, or `tall` so they can take more space in the grid.
6. **Turn the dashboard into a Grid:**
   Use `display: grid`, `grid-template-columns: repeat(3, 1fr)`, and `gap` to create a clean three-column layout.
7. **Control automatic rows:**
   Use `grid-auto-rows` with a comfortable height so automatically created rows do not look too small or uneven.

8. **Position special cards:**
   Use `grid-column: span 2` or `grid-row: span 2` for featured cards to make the dashboard look more dynamic.
9. **Explore auto-placement:**
   Use `grid-auto-flow: row` first and observe how the cards are placed automatically.
10. **Try dense placement:**
    Change it to `grid-auto-flow: row dense` and notice how the browser tries to fill empty spaces. Students should write one sentence explaining whether dense placement helped or made the order confusing.
11. **Use Flexbox inside cards:**
    Use Flexbox inside each card to align the content vertically and push the action link to the bottom.
12. **Use Grid Inspector:**
    Open DevTools, enable Grid Inspector, and check the columns, row tracks, gaps, and how large cards are spanning across the layout.
13. **Final comparison:**
    Add a short comment in the CSS file explaining why Grid was used for the dashboard layout and Flexbox was used inside the cards.

Duration: 35 minutes